SMART CAMPUS

Technical deck · v1

SmartCampus-CentralServer

Architecture du serveur central, protocole WebSocket et authentification mutuelle des appareils par clés publiques.

Liam COLLE · TEK 3 – EPITECH Marseille · Bureau d'innovation du Hub · Octobre 2026

PÉRIMÈTRE

Le serveur central orchestre tout le campus

Administration

API REST et canal temps réel pour la console React.

Gateway

Connexions WSS des appareils, handshake mutuel, registre en ligne / hors ligne.

Orchestration

Commandes, statuts de salles, annonces, audio InterCom, planification.

Persistance

MongoDB : référentiel, état des appareils, fichiers et journal d'audit.

Appareils enfants : Door unit · Famoco, React Native · Faculty display · mini-PC ou Raspberry Pi · Access unit · Raspberry Pi + lecteur de badges

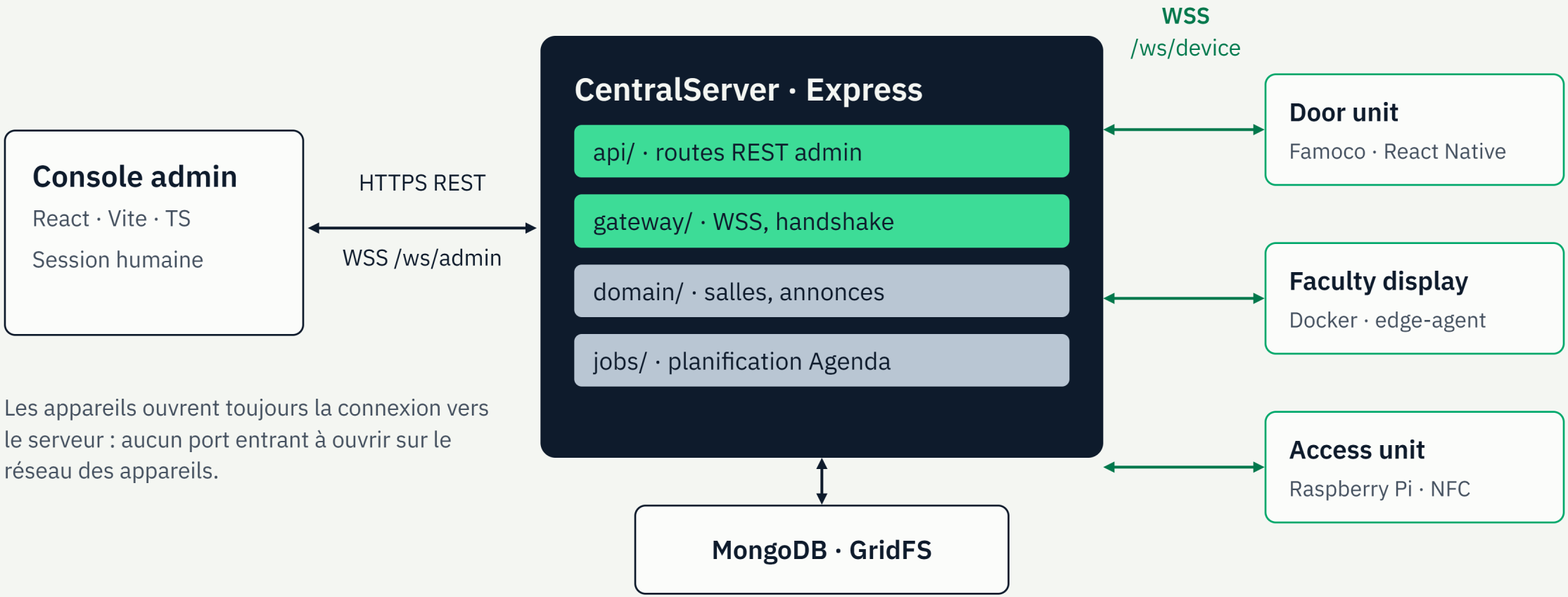
STACK

La base existante, complétée de cinq briques

Couche	Technologie	Statut	Rôle
Console admin	React · Vite · TypeScript	Existant	Interface d'administration
API	Express · TypeScript	Existant	REST et upgrade WebSocket sur le même serveur
Base de données	MongoDB · Mongoose	Existant	Référentiel, état des appareils, annonces
Temps réel	ws (WSS)	Proposé	Contrôle total du handshake, frames binaires
Validation	zod	Proposé	Schémas de messages partagés serveur / appareils
Cryptographie	node:crypto	Proposé	ECDSA P-256, ECDH, HKDF, AES-256-GCM natifs
Planification	Agenda	Proposé	Jobs persistés dans MongoDB, sans Redis
Logs	pino	Proposé	Logs JSON structurés, corrélés par appareil

ARCHITECTURE

Un serveur, deux canaux WSS



Les appareils ouvrent toujours la connexion vers le serveur : aucun port entrant à ouvrir sur le réseau des appareils.

ORGANISATION DU CODE

Un monorepo, un contrat de messages partagé

Monorepo · npm workspaces

apps/web	React · Vite (existant)
apps/server	Express (existant)
apps/door	React Native · Famoco
apps/edge-agent	Node TS · écrans, accès
packages/protocol	types + schémas zod
packages/handshake	crypto Node et RN

apps/server/src

server.ts	HTTP + routage de l'upgrade WSS
api/	routes REST de la console
gateway/	wss · handshake · registry · router
domain/	devices · campus · announcements · intercom · access
jobs/	planification (Agenda)
infra/	mongo · gridfs · logger

packages/protocol est importé par le serveur, l'app Famoco et l'edge-agent : un seul contrat de messages, vérifié à la compilation (TypeScript) et à l'exécution (zod).

MODÈLE DE DONNÉES

Huit collections MongoDB

Collection	Contenu clé	Index
users	Comptes de la console, rôles, hash du mot de passe	email unique
devices	Type, clé publique, empreinte, statut, emplacement, groupes, état désiré / reporté, lastSeen	empreinte unique, groupes
rooms	Bâtiment, étage, statut courant de la salle	buildingId
announcements	Texte, audio, cibles, priorité, planning, expiration	startsAt
commands	Outbox : appareil, type, payload, statut, expiresAt	TTL expiresAt, deviceId + statut
assets	Fichiers audio et images (GridFS) avec empreinte SHA-256	sha256
accessRules	Badges, zones, plages horaires	badgeUid
auditLogs	Qui a fait quoi, quand, sur quel appareil	ts

Partie 2

Comment les appareils parlent au serveur

Transport, modèle de connexion, protocole de messages et garanties de livraison.

PROPOSITION · TRANSPORT

Trois options de transport

Critère	A · ws + protocole maison	B · Socket.IO	C · MQTT
Infrastructure	Aucune, même serveur Express	Aucune, même serveur Express	Broker à opérer (Mosquitto)
Handshake à clés	Contrôle total	Possible via middleware	Délégué au broker
Audio, binaire	Frames binaires natives	Supporté	Supporté
Client Famoco	WebSocket natif de React Native	socket.io-client	Bibliothèque MQTT
Reconnexion, acks	À écrire nous-mêmes	Intégrés	Intégrés (QoS)
Verdict	Recommandée	Alternative solide	Écartée à cette échelle

A garde la main sur le handshake et le chiffrement applicatif (partie 3). B reste un repli si le temps manque ; C devient pertinent avec des milliers d'appareils.

MODÈLE DE CONNEXION

Une connexion persistante par appareil

Sortante

L'appareil ouvre la connexion WSS. Aucun port entrant à ouvrir sur l'appareil.

Unique

Registre deviceId → socket. Une nouvelle connexion remplace l'ancienne.

Surveillée

Ping toutes les 25 s. Deux pongs manqués : hors ligne et alerte console.

Résiliente

Backoff de 1 à 60 s avec aléa, pour éviter une reconnexion massive simultanée.

Jumeau numérique Le serveur stocke l'état **désiré** (ce que l'admin veut) et l'état **reporté** (ce que l'appareil affiche). À chaque reconnexion, il renvoie l'état désiré complet : l'appareil converge, même après une longue coupure.

PROTOCOLE

Une enveloppe JSON, des types nommés

```
{
  "v": 1,
  "id": "01J9ZQ3K…",
  "type": "room.status.set",
  "ts": 1760011200000,
  "seq": 42,
  "payload": {
    "roomId": "B2-104",
    "status": "occupied"
  }
}
```

id : ULID, pour l'idempotence · **seq** : compteur, contre le rejeu

Sens	Type	Rôle
S → A	room.status.set	Statut de salle
S → A	display.config.set	Mise en page
S → A	announce.show	Annonce visuelle
S → A	audio.play	Annonce audio
S → A	access.acl.sync	Liste des badges
A → S	ack · nack	Accusé de réception
A → S	state.report	État, télémétrie
A → S	event.badge	Badge présenté

S = serveur · A = appareil · chaque type a son schéma zod dans packages/protocol.

GARANTIES DE LIVRAISON

Aucune commande perdue ni doublée

Outbox

Chaque commande est écrite dans **commands** avant l'envoi : pending → sent → acked, ou expired (index TTL).

Idempotence

L'appareil garde les derniers id traités : une commande renvoyée n'est jamais exécutée deux fois.

Reprise

À la reconnexion : l'état désiré complet, puis les commandes non expirées, dans l'ordre.

▷ **Audio jamais dans le JSON.** audio.play transporte un assetId, son SHA-256 et sa taille. L'appareil télécharge le fichier en HTTPS avec une requête signée par sa clé (méthode, chemin, horodatage), vérifie l'empreinte, puis le garde en cache.

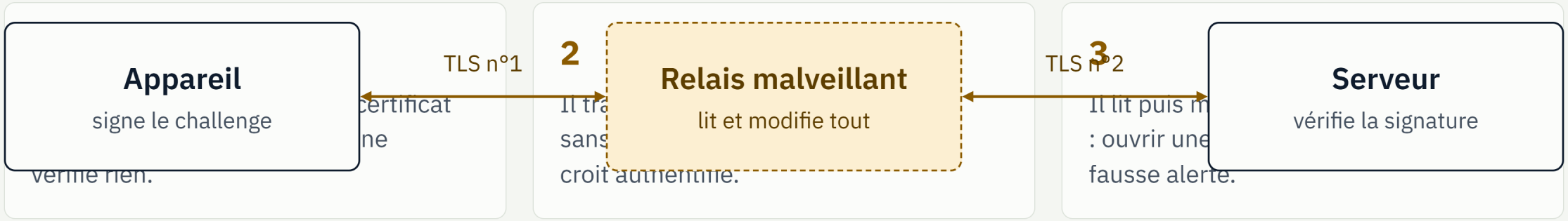
Partie 3

Identité des appareils et sécurité des échanges

WSS sans vérification du certificat et authentification mutuelle par clés : ce qu'il faut ajouter pour que l'ensemble reste sûr.

POINT D'ATTENTION

Le risque : un relais au milieu



La preuve d'identité doit être liée au canal : clé de session signée (P2) ou épinglage du certificat (P3).

PROPOSITION · SÉCURITÉ

Quatre options, deux à combiner

Proposition	Principe	Relais	Avis
P1 · Challenge signé	Chaque côté signe un nonce de l'autre	Non	Insuffisant seul
P2 · Clé de session signée	ECDH éphémère signé par les deux clés, messages chiffrés AES-256-GCM	Oui	Recommandé : sûr même si TLS est relayé
P3 · Épinglage	L'appareil n'accepte que le certificat auto-signé connu	Oui	Recommandé en complément, coût faible
P4 · mTLS	Certificats clients vérifiés par TLS	Oui	Écarté : impose la vérification de certificats

Sur Famoco, React Native refuse en principe un certificat auto-signé : le déclarer dans le **network_security_config** Android règle ce point et réalise P3.

P2 EN DÉTAIL

Le handshake en quatre messages



`T = deviceId ‖ nonceA ‖ nonceS ‖ ephA ‖ ephS`

sigS : le serveur signe T ; l'appareil vérifie avec la clé serveur provisionnée.

sigA : l'appareil signe T ; le serveur vérifie avec la clé enregistrée à l'enrôlement.

Session = HKDF-SHA256(ECDH(ephA, ephS), T), puis AES-256-GCM.

Nonces de 32 octets, délai de 10 s, clés éphémères jetées.

GESTION DES CLÉS

Enrôlement, stockage, révocation

Enrôler un appareil

1. Au premier démarrage, l'appareil génère sa paire de clés. La clé privée ne sort jamais.
2. Il affiche un code court et l'empreinte de sa clé publique.
3. Un admin saisit ce code dans la console et affecte l'appareil à une salle.
4. La clé publique est enregistrée dans devices : le handshake normal devient possible.

Où vivent les clés

Où	Stockage
Famoco	Android Keystore, non exportable
Pi, mini-PC	Fichier 0600 hors image, ou TPM
Serveur	Secret monté, hors du dépôt
Clé du serveur	Provisionnée par Famoco DMS / image

Révocation en un clic : socket fermé immédiatement, handshake refusé ensuite. **Rotation** : nouvelle paire de clés, puis ré-enrôlement.

APPAREILS ENFANTS

Trois clients, un même protocole

Door unit

Famoco · React Native

- WebSocket natif, certificat épinglé
- Clé dans Android Keystore via un petit module natif
- Statut, annonces et audio InterCom
- Déployée par Famoco DMS avec la clé du serveur

Faculty display

mini-PC ou Raspberry Pi · Docker

- edge-agent Node TS : clé et connexion WSS
- Chromium en kiosque affiche l'app locale
- L'agent pousse l'état à la page via ws://localhost
- Le navigateur ne voit jamais la clé

Access unit

Raspberry Pi · lecteur NFC

- Même edge-agent, module access
- Lecteur de badges et relais de gâche
- Liste d'accès en cache : décide même hors ligne
- Événements mis en file, puis remontés

⚠ **À valider :** en cas de coupure, la porte reste-t-elle verrouillée ou s'ouvre-t-elle ? À décider avec le responsable sécurité incendie.

EXPLOITATION

Simple à déployer, prêt à grandir

Déploiement

- Docker Compose : server et mongo
- TLS terminé par Node, certificat auto-signé
- Configuration par variables d'environnement
- Logs pino et route /health

Montée en charge

- Une instance suffit pour un campus
- Plus tard : plusieurs instances et Redis pub/sub
- Redis route chaque commande vers l'instance qui tient le socket

Durcissement

- Taille maximale des messages (maxPayload)
- Limite de débit sur handshake et enrôlement
- Schéma zod vérifié sur chaque message
- Journal d'audit des actions sensibles

PLAN D'IMPLÉMENTATION

Aligné sur les quatre soutenances

9 oct.

Socle

- Gateway WSS et registre
- Enrôlement et handshake P2
- packages/protocol

23 oct.

Door unit

- Outbox et accusés
- Statut de salle, annonces
- audio.play, requêtes signées

6 nov.

Faculty

- edge-agent Docker et kiosque
- Annonces planifiées (Agenda)
- Voix générée par IA

20 nov.

Accès, pilotage

- Access unit : liste d'accès, événements
- Diffusion d'urgence
- Tableau de bord, audit

La gateway et le handshake passent en premier : toutes les autres fonctionnalités en dépendent.

À TRANCHER

Cinq décisions avant de coder

Décision	Options	Proposition
Algorithme d'identité	P-256 ou Ed25519	P-256, sauf si tous les Famoco ont un Android récent
Transport	ws ou Socket.IO	ws, avec Socket.IO en repli
Matériel des écrans	mini-PC x86 ou Raspberry Pi	Images Docker multi-architecture (amd64 et arm64)
Porte en cas de coupure	Verrouillée ou ouverte	À valider avec la sécurité incendie
Synthèse vocale	Service cloud ou modèle local	Interface TTS abstraite, choix du moteur plus tard